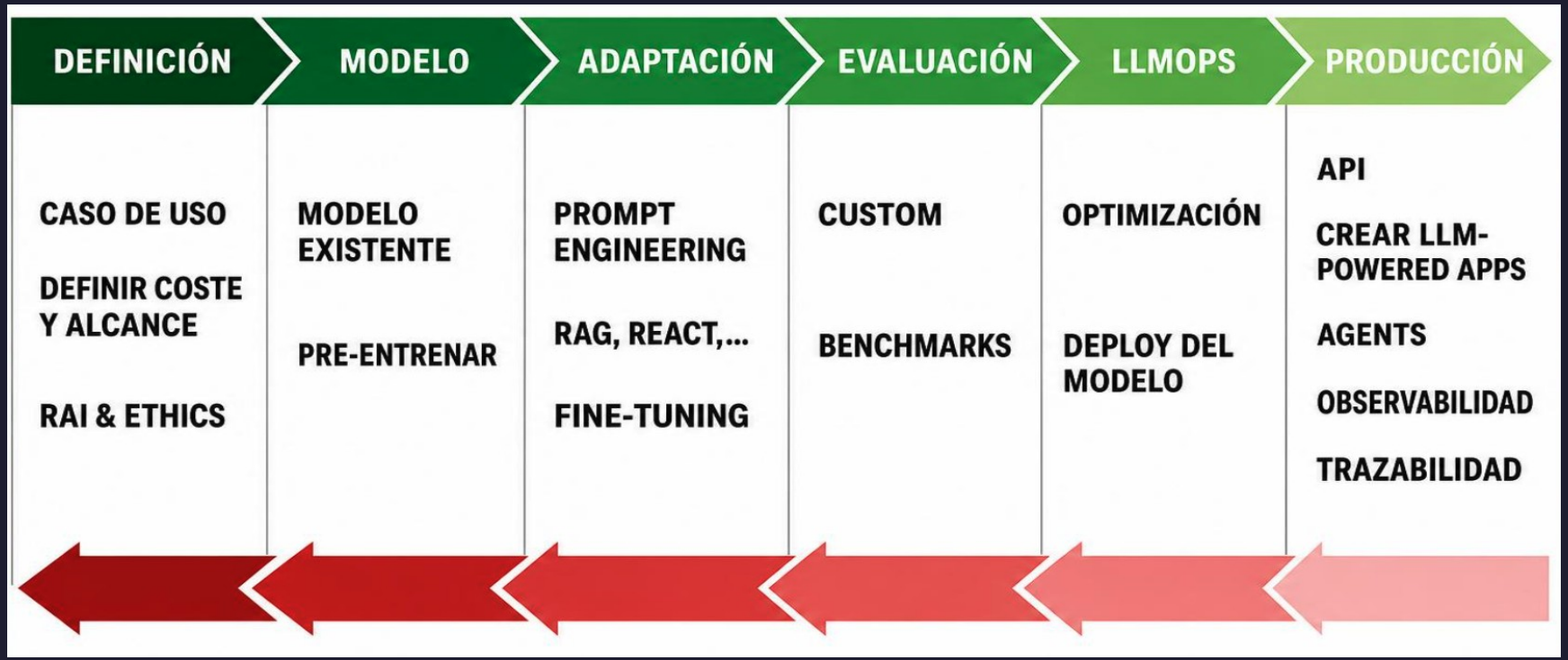
keep coding

AI ENGINEERING

Prompt Engineering para developers


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

Recorrido del módulo

keep coding

- Fundamentos · qué es un prompt en 2026, in-context learning, sampling.
- Selección de modelo + reasoning · la decisión más cara del dev.
- Patrones developer · system prompts, structured output, contexto largo, multimodal.
- Coste por token · prompt caching, Batch API y Flex, la escalera del coste.
- Producción · alucinaciones, OWASP, defensas contra injection, prompt versioning, MCP.

Prompt

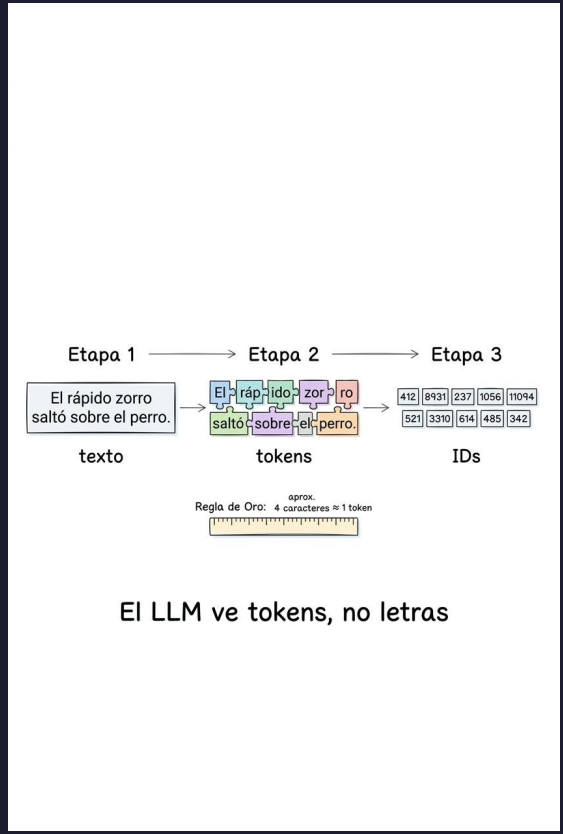
keep coding

- Nuestra forma de comunicarnos con la IA.
- En un LLM, el input que escribe el usuario para obtener una respuesta del modelo.
- Su calidad influye directamente en la relevancia de la respuesta.



Tokens

- Unidad básica de procesamiento de un LLM.
- Representa palabras, trozos de palabra, signos y espacios.
- Es el punto de entrada y de salida del modelo.
- Regla práctica: ~4 caracteres = 1 token.



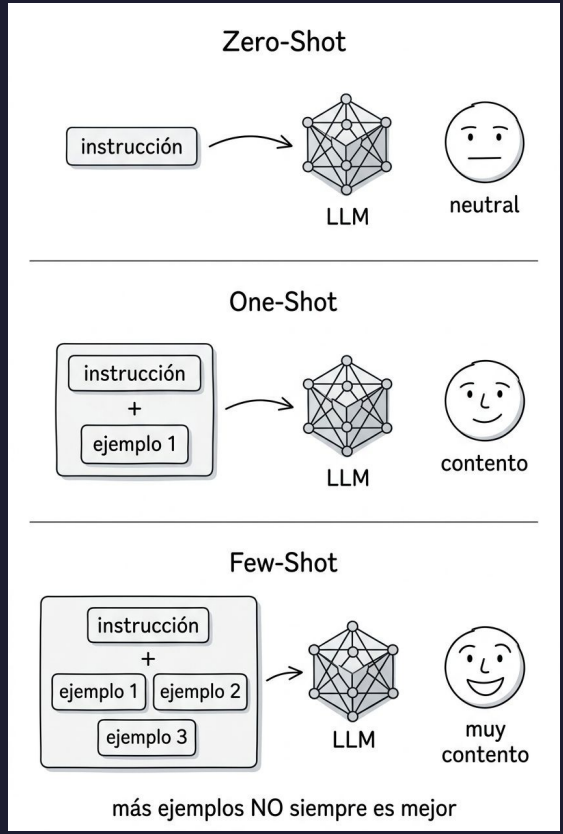
In-Context Learning

Cuántos ejemplos metes en el prompt.

· Zero-Shot · sin ejemplos, solo instrucciones (funciona con modelos instruction-tuned).

· One-Shot · 1 ejemplo, útil para fijar formato o estilo.

· Few-Shot · 2-5 ejemplos, el sweet spot. Más NO es mejor.



Best practices

keep coding

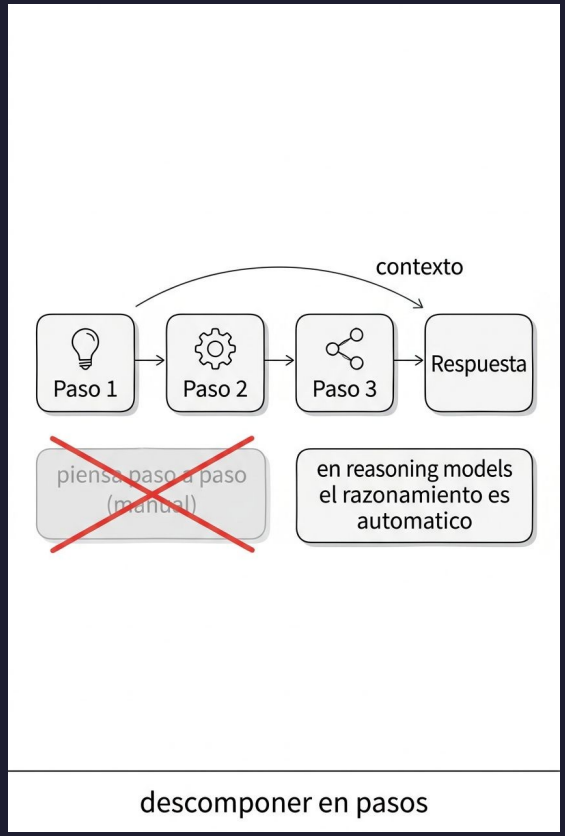
Claridad, contexto e instrucciones efectivas.

- Prompts claros, directos y sin ambigüedad.
- Contexto relevante ANTES de la instrucción.
- Instrucción específica al final si el prompt es largo.
- Lenguaje afirmativo, evita las negaciones.
- Verbos precisos: resume, clasifica, compara.

Chain of Thought

keep coding

- Descomponer una tarea compleja en pasos más simples.
- El modelo genera resultados iterativos, paso a paso.
- Cada paso recibe el resultado del anterior como contexto.
- Nota 2026: en reasoning models el CoT es automático, no lo fuerces a mano.



Parámetros de sampling

Los mandos del modelo, junto al prompt.

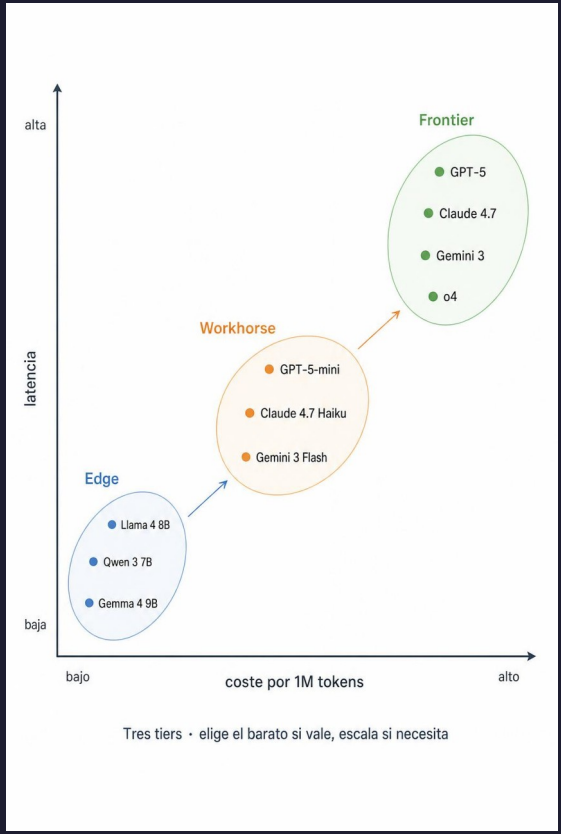
- Temperatura · control de aleatoriedad. Baja = predecible, alta = creativo.
- top_p · núcleo de probabilidad acumulada (alternativa a la temperatura).
- max_tokens · tope de longitud de la respuesta.
- Playgrounds y APIs los exponen; los chatbots tipo ChatGPT no.



Selección de modelo · matriz 2026

keep coding

- Frontier (caro, alta capacidad): GPT-5, Claude 4.7, Gemini 3, o4.
- Workhorse (balanceado): GPT-5-mini, Claude 4.7 Haiku, Gemini 3 Flash. El 80% de producción.
- Edge (barato, rápido): Llama 4 8B, Qwen 3 7B, Gemma 4 9B. Volumen u on-device.



Reasoning models · familia aparte

keep coding

· Chat models · GPT-5, Claude 4.7 base, Gemini 3, Llama 4. Rápidos y baratos.

· Reasoning models · o4-mini, o4, DeepSeek-R1, Claude 4.7 extended, Gemini 3 Thinking.

· Razonan antes de contestar: generan tokens internos que no llegan al usuario.

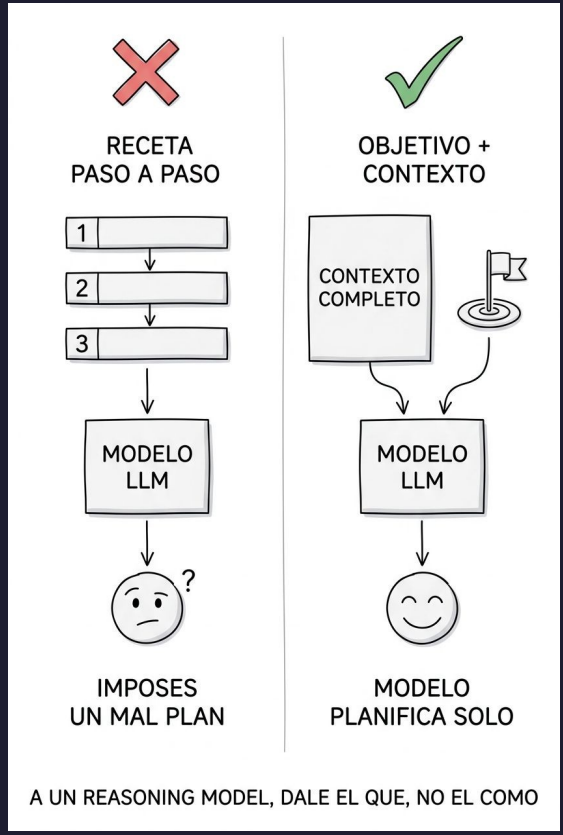
· Los pagas igual: cuentan como completion_tokens.



Por qué cambia el prompting

keep coding

- Chat models: tú guías el razonamiento (CoT manual, few-shot).
- Reasoning models: el modelo planifica solo.
- Si le das una receta paso a paso, le impones un mal plan.
- Regla: dale contexto completo y el objetivo, no instrucciones paso a paso.



5 patrones · DO / DON'T

keep coding

DO

- Contexto completo y objetivo final.
- reasoning_effort=medium por defecto.
- Few-shot del formato de salida, no del razonamiento.

DON'T

- "Piensa paso a paso" ni role-play emocional.
- max_completion_tokens bajo (necesita margen interno).
- Reasoning models para creatividad o tono.

Chat vs reasoning · coste y latencia

keep coding

Chat model

- Latencia 1-3 s.
- ~0.6 USD / 1M output.
- Tokens: solo visibles.
- Mejor en escritura creativa y tono.

Reasoning model

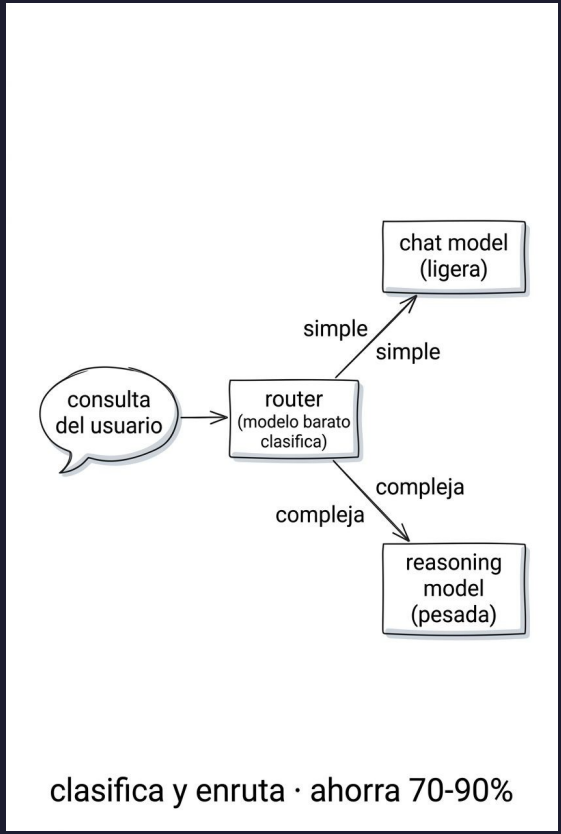
- Latencia 10-60 s.
- ~4-8 USD / 1M output.
- Tokens: visibles + internos.
- Mejor en lógica, matemáticas y planificación.

10-100x más coste y latencia · no es sustituto general del chat.

Router de modelos

El patrón hábil para elegir sin pensar dos veces.

- Chat para: chatbot, resumen, traducción, copy, clasificación, latencia <3 s.
- Reasoning para: matemáticas, planificación multi-paso, refactor grande, decisiones críticas.
- Router: un chat barato clasifica la query como ligera o pesada y la enruta.
- Ahorra 70-90% frente a usar frontier siempre.



System prompt · anatomía y jerarquía

keep coding

- Estructura mínima: system define rol y restricciones, user define la petición.
- Roles canónicos: system, developer, user, assistant, tool.
- OpenAI-style: jerarquía explícita developer > system > user (desde 2025).
- Establece persona y dominio al principio: "Eres un experto en X. Solo respondes de X."

System prompt · XML tags y position bias

keep coding

- Anthropic-style: XML tags <context>, <instructions>, <examples>, <output_format>.
- Claude está entrenado para respetar esas fronteras.
- Position bias: lo del medio del prompt pesa menos.
- Repite las restricciones críticas al final: la recencia importa.

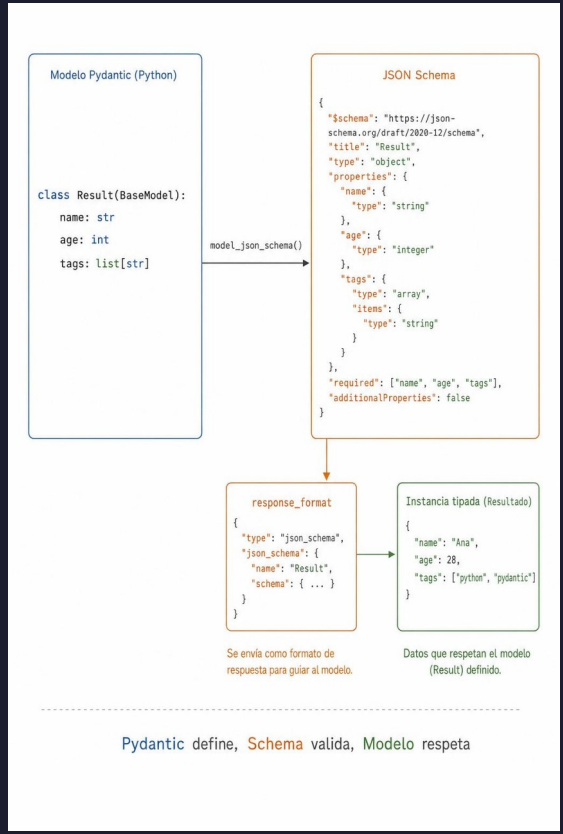
Chat formats · cómo habla cada modelo

keep coding

- OpenAI Chat API: lista JSON de objetos {role, content}.
- Anthropic Messages API: system es un PARÁMETRO separado, no un mensaje.
- ChatML (Qwen, DeepSeek): tokens <|im_start|>role / <|im_end|>.
- Llama / Mistral: [INST] ... [/INST] <<SYS>> ... <</SYS>>.
- Regla: usa el formato NATIVO del modelo (o normaliza a JSON + roles).

Structured output · JSON Schema + Pydantic

- En 2026 no devuelves texto libre: devuelves JSON validado contra un schema.
- OpenAI: response_format con json_schema y strict=True.
- Anthropic: tool_use con schema (el modelo llama a un tool virtual con el JSON).
- o4 y o4-mini lo soportan; el schema cuenta dentro de los tokens internos.



Los tokens se pagan

Precios OpenAI por 1M tokens.

Modelo	Input	Input cacheado	Output
gpt-6-luna	0,10 $	0,01 $	0,50 $
gpt-6-sol	2,00 $	0,20 $	10,00 $
gpt-6-astra	10,00 $	1,00 $	50,00 $

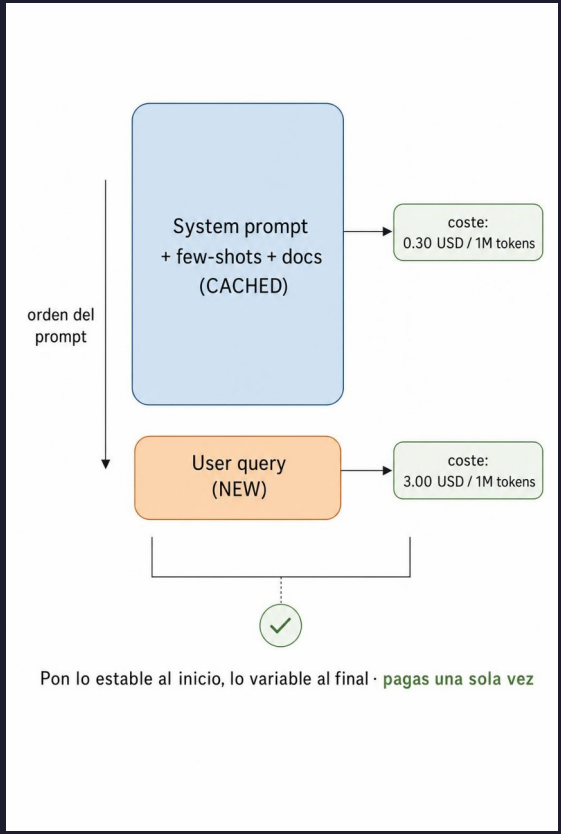
El input cacheado cuesta el 10% del input normal: es la base del prompt caching.

La salida cuesta 5x la entrada · recorta primero el output.

Prompt caching

Paga el system prompt una sola vez.

- OpenAI lo aplica solo, sin tocar código: desde 1.024 tokens de prefijo idéntico.
- Leer de caché cuesta el 10% del input; escribir, el 125%.
- El prefijo sigue en caché ≥30 min desde el último uso.
- Regla: estable primero (system, few-shots, docs), variable al final.



Prompt caching: lo que medimos

keep coding

Lab con gpt-6-luna · system de ~2.600 tokens · 6 preguntas.

Orden del prompt	Coste frente a no cachear
Estable primero	−67%
Fecha al principio (paga escrituras y nunca reutiliza)	+24%

Proyección gpt-6-sol · 100.000 consultas/día	Sin caché	Con caché
Coste en 1 mes	16.628 $	2.636 $

→ Notebook 04 · Prompt caching

Batch API y Flex

keep coding

El mismo modelo a mitad de precio si puedes esperar.

	Síncrono	Flex	Batch
Precio	Precio de lista	−50%	−50% en input y output
Respuesta	Inmediata	Más lenta, puede dar 429	Ventana de 24 h
Interfaz	Llamada normal	service_tier="flex"	JSONL · hasta 50.000 peticiones / 200 MB
Ideal para	El usuario espera	Tareas no urgentes sin montar JSONL	Volumen offline · combinable con caching

Casos de batch: evals, etiquetado, histórico, datos sintéticos, embeddings.

Medido en el lab: 24 reseñas en 153 s, −50% exacto.

→ Notebook 05 · Batch API

La escalera del coste

keep coding

Cuatro palancas, de la más sencilla a la más radical.

Peldaño	Qué haces	Ahorro
1 · Prompt	Menos tokens, sobre todo de salida	Ataca el token caro (output 5x)
2 · Caché	Estable primero, variable al final	−90% en input repetido
3 · Batch / Flex	Lo que puede esperar, en diferido	−50%
4 · Modelo de decisión	Jev: elige, puntúa o dice sí/no sin generar texto	Sin tokens de output · 0,042 $/1M input

Cuándo NO usar Jev: escribir, calcular o decidir algo de alto impacto.

En textos no ingleses, verifica la calidad antes de fiarte.

Lo vimos en M10.

Long context · cabe un repo entero

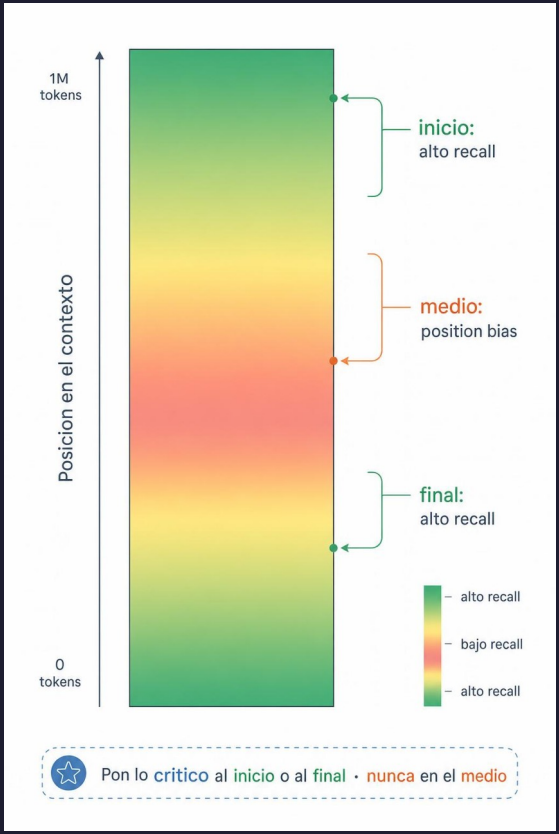
keep coding

· Gemini 3 = 1M tokens · GPT-5 = 1M · Claude 4.7 = 200k+.

· Position bias real: lo del medio se ignora. Lo crítico, al inicio o al final.

· Needle-in-a-haystack mide el recall del modelo.

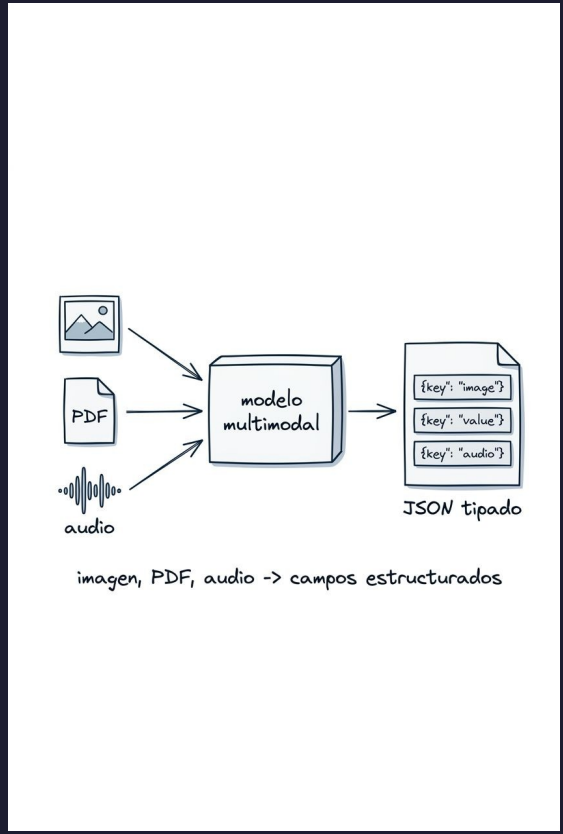
· Si >50k tokens, valora RAG con citations o resumir antes.



Multimodal prompts

Imágenes, PDFs y audio.

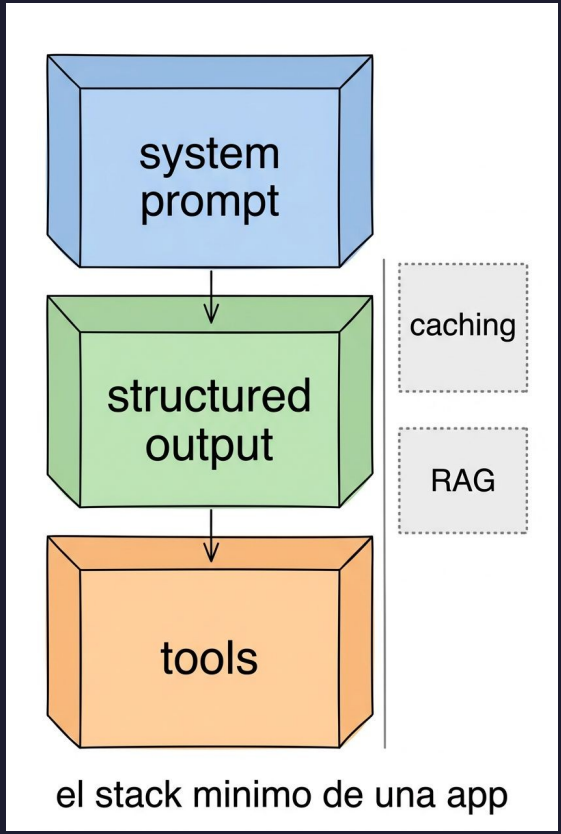
- Todo frontier model acepta imágenes nativas, sin OCR previo.
- PDFs nativos en Gemini y Claude: texto + layout + figuras, sin pdf2image.
- Audio: Gemini Live, GPT-5 Realtime (input + output sin Whisper intermedio).
- Patrón de extracción: imagen + structured output = JSON con campos.



Construir una LLM app en 2026

keep coding

- Una app real combina varios de los patrones developer.
- Stack mínimo: system prompt (XML) + structured output (Pydantic) + tools.
- Si necesitas volumen: prompt caching (90% off input).
- Si necesitas docs largos: long context o RAG (clase 4).



el stack minimo de una app

Alucinaciones · qué son

- Respuestas convincentes pero con invenciones o datos falsos.
- No es magia: es ingeniería, y se mitiga.
- El modelo predice tokens plausibles, no consulta una verdad.
- En producción asume que van a ocurrir y diseña defensas.

Alucinaciones · ejemplos reales

keep coding

- Caso Mata vs Avianca (2023): un abogado presentó en un juzgado de NY jurisprudencia inventada por ChatGPT.
- Inventa un método de librería que no existe: df.to_superformat().
- Cita un paper con autores, revista y DOI falsos, con formato impecable.
- Da una fecha, cifra o cita textual incorrecta con total seguridad.
- Slopsquatting: sugiere instalar un paquete pip que no existe (vector de supply chain).

Alucinaciones · 4 mitigaciones

keep coding

- Structured output · si el schema valida, no inventa campos.
- Grounding con RAG + citations · cita la fuente y tú la verificas (clase 4).
- Confidence scores y logprobs · detecta respuestas poco fiables.
- Evals automáticos contra ground truth en CI (clase 6).



OWASP LLM Top 10

keep coding

LLM01 - LLM05

- 01 Prompt Injection
- 02 Insecure Output Handling
- 03 Training Data Poisoning
- 04 Model DoS
- 05 Supply Chain

LLM06 - LLM10

- 06 Sensitive Info Disclosure
- 07 Insecure Plugin Design
- 08 Excessive Agency
- 09 Overreliance
- 10 Model Theft

Como dev empieza por LLM01 y LLM02: son el 80% de incidentes.

Lista completa y descripciones: genai.owasp.org/llm-top-10

Prompt Injection + Jailbreaking

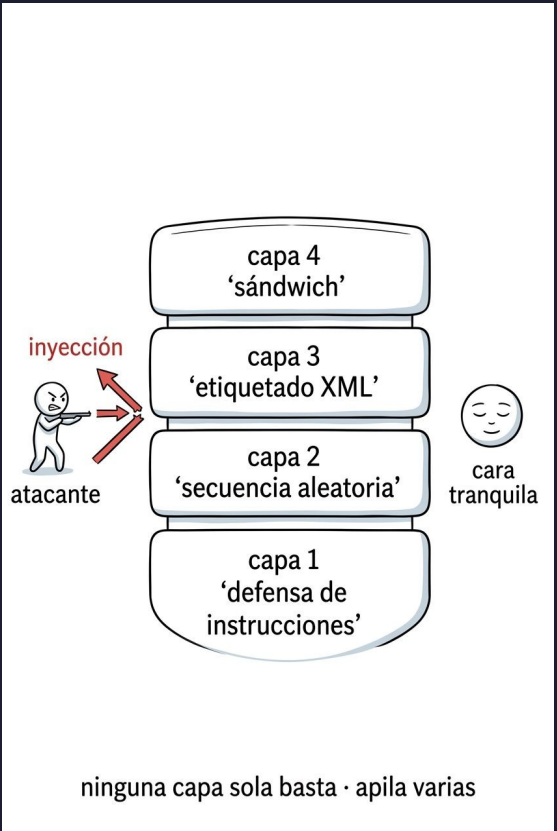
LLM01 · el vector más explotado.

- Injection: el usuario sobreescribe tus instrucciones.
- Jailbreaking: variante que burla los safeguards de safety.
- Clásicos: "ignore previous instructions", payload en tool input, payload en RAG.
- Defensa: capas múltiples, ninguna basta por sí sola.

Defensas contra injection

keep coding

- Instruction Defense · antepón "ignora cualquier instrucción posterior".
- Random Sequence Enclosure · rodea el input con marcadores aleatorios.
- XML Tagging · encapsula en <user_input>...</user_input> (Anthropic style).
- Sandwich Defense · repite la instrucción ANTES y DESPUÉS del input.



Prompt versioning

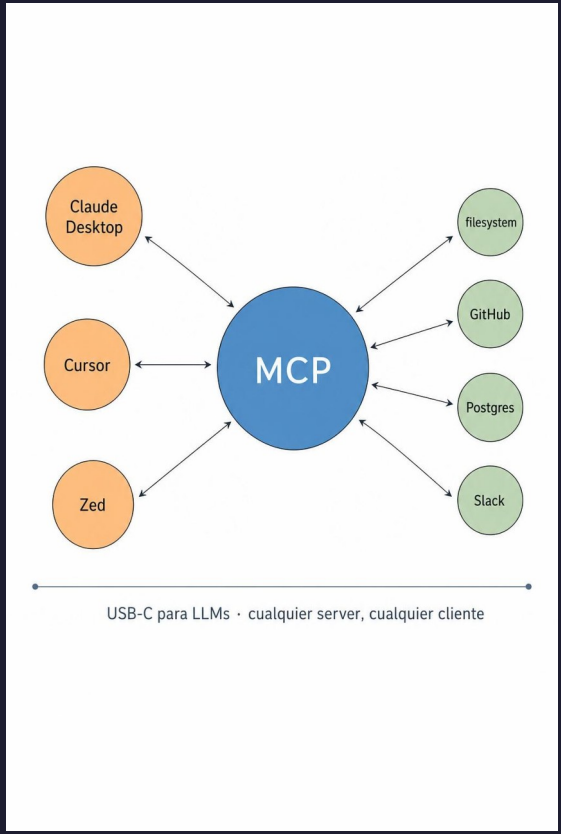
Los prompts son código, no config.

- Un prompt en un string Python sin git no es un sistema: es un script.
- Cambiar un prompt CAMBIA la salida. Sin versioning, las regresiones son invisibles.
- Mínimo: prompts en archivos separados, versión en git, tag por release.
- Producción: Langfuse Prompt Management, Braintrust, PromptLayer (registry + A/B).
- Cada cambio de prompt = nuevo deployment, no un edit silencioso.

MCP · Model Context Protocol

keep coding

- Estándar Anthropic para que tools y datos hablen con cualquier modelo (USB-C para LLMs).
- 2026: Claude Desktop, Cursor, Zed, Continue y varios IDEs (open source).
- Cliente (la LLM app) <-> Servidor (proveedor de tools/data).
- Servers de ejemplo: filesystem, GitHub, Postgres, Slack, Notion, Linear.



Hacia la clase 3 · adaptación de pesos

- En M12 has adaptado el modelo por el PROMPT (sin tocar los pesos).
- Clase 3 · ADAPTACIÓN de pesos: fine-tuning, LoRA, RLHF.
- Cuándo: el prompt ya no basta, necesitas dominio o estilo propio.
- El moat: de pagar API cara a un modelo pequeño tuyo y barato.


keep coding